

Operational Hardening

From "works on stage" to "runs every night, unattended"

What this chapter is about

So far we've built a discovery pipeline that *works* — once. Operational hardening is the difference between "the demo went great" and "it has been running for a customer for 11 months and never woken anyone up at 2am." This chapter is about scheduled re-discovery, alerting on changes, observability of our own pipeline, polite retries, and graceful failure. Less glamorous than the previous chapters, but where most home-grown systems quietly die.

Real-life analogy. Building the map (Chapters 1–4) is like designing and constructing a brand-new shopping mall. Operational hardening is the night shift — the watchman who walks the corridors every two hours, the cleaners, the alarm system, the CCTV, the maintenance log, the duty manager who gets called only when something is genuinely on fire (not every time a stray cat triggers a sensor). Without the night shift, the mall opens once and slowly falls apart.

Words you'll see (and what they mean)

Cadence. How often a recurring task runs. Different pillars have different natural cadences — pinging a switch every minute is fine; doing a full subnet port-scan every minute is reckless.

Cron / cron job. A scheduled task that runs on a fixed timing pattern. The classic Linux cron syntax is `* * * * *` (minute hour day month weekday). We use the Go library github.com/robfig/cron/v3 in the backend and (separately) [@nestjs/schedule](https://github.com/nestjs/schedule) in NestJS.

Backoff. When a target fails, don't retry immediately — wait, then retry, then wait longer, etc. *Exponential* backoff: 5s, 10s, 20s, 40s ... capped at some maximum. *Jitter*: add random $\pm 25\%$ so a hundred retries don't all fire at the exact same second. We use github.com/cenkalti/backoff/v4.

Worker pool. A fixed-size set of goroutines that pick jobs off a queue. Stops you from accidentally spawning 50,000 goroutines and crashing your collector. We use github.com/panjf2000/ants/v2.

Hysteresis. "Be slow to change my mind." We don't flip a device up→down on one missed ping; we wait for 3 in a row. Prevents flapping when a single packet is lost.

Flapping. A device or link rapidly oscillating between `up` and `down`. Almost always a symptom of either a real flaky cable or insufficient hysteresis.

Prometheus. An open-source metrics-collection system. We expose internal counters like `scan_jobs_completed_total` on a `/metrics` endpoint; a Prometheus server scrapes them every 15 s, and we visualise them in Grafana.

SLO (Service Level Objective). A target like "99% of crawl jobs complete in < 30 minutes" or "no scan job is queued longer than 1 hour." Lets the team see when the system is silently degrading.

Structured logging. Logs as JSON objects (level, time, msg, fields), not free-form strings. Lets a log aggregator filter by, e.g., `tenant_id=acme` instantly. We use `go.uber.org/zap`.

Webhook. An HTTP callback. Instead of "we tell the customer", we "POST a JSON payload to a URL the customer gave us when something happens." Standard pattern for alert delivery.

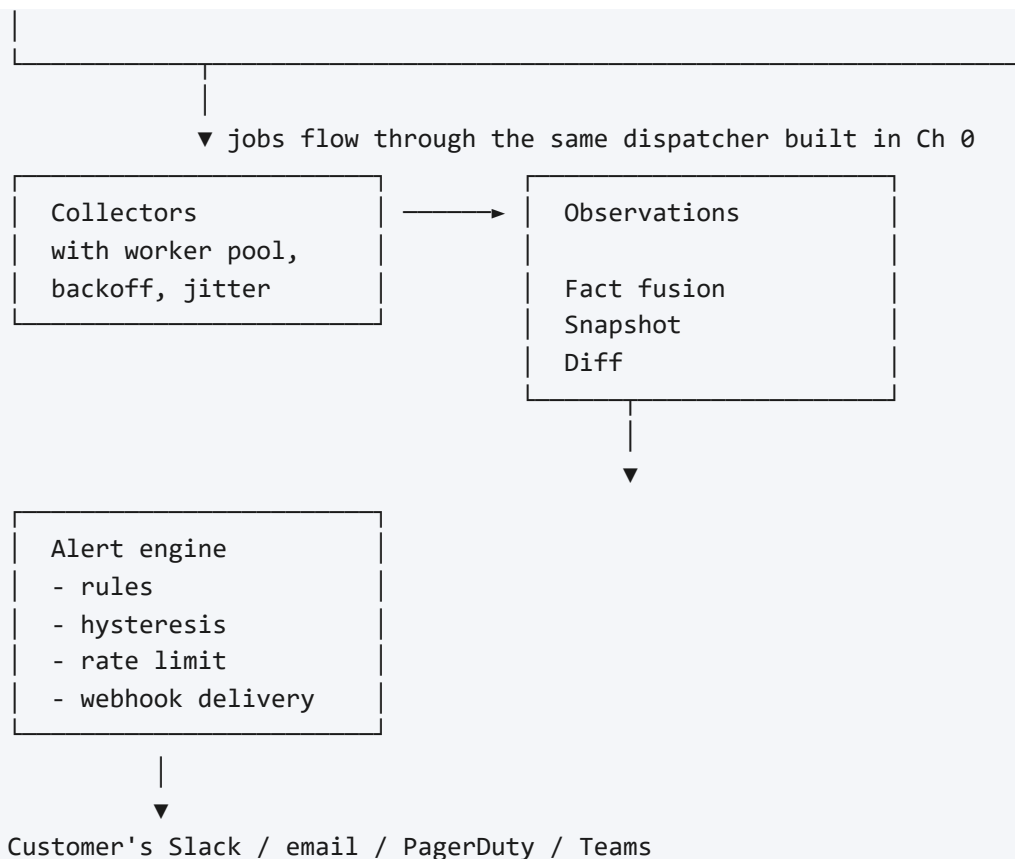
Drift. A configuration change you didn't make. Topology drift = "yesterday switch-A had 22 hosts; today it has 24 – was that you?" These are exactly the changes that need surfacing.

What we're building (the picture)

```

Server side cron
every 5 min → enqueue ICMP liveness sweep per site
every 1 hr  → enqueue SNMP inventory walk per device
every 24 hr → enqueue full active scan per tenant CIDR
every 6 hr  → enqueue firewall config ingest
every 1 hr  → enqueue topology snapshot
every 15 min → sweep stale-job timeouts
every 1 day → lifecycle sweep (active→decom; decom→archive)

```



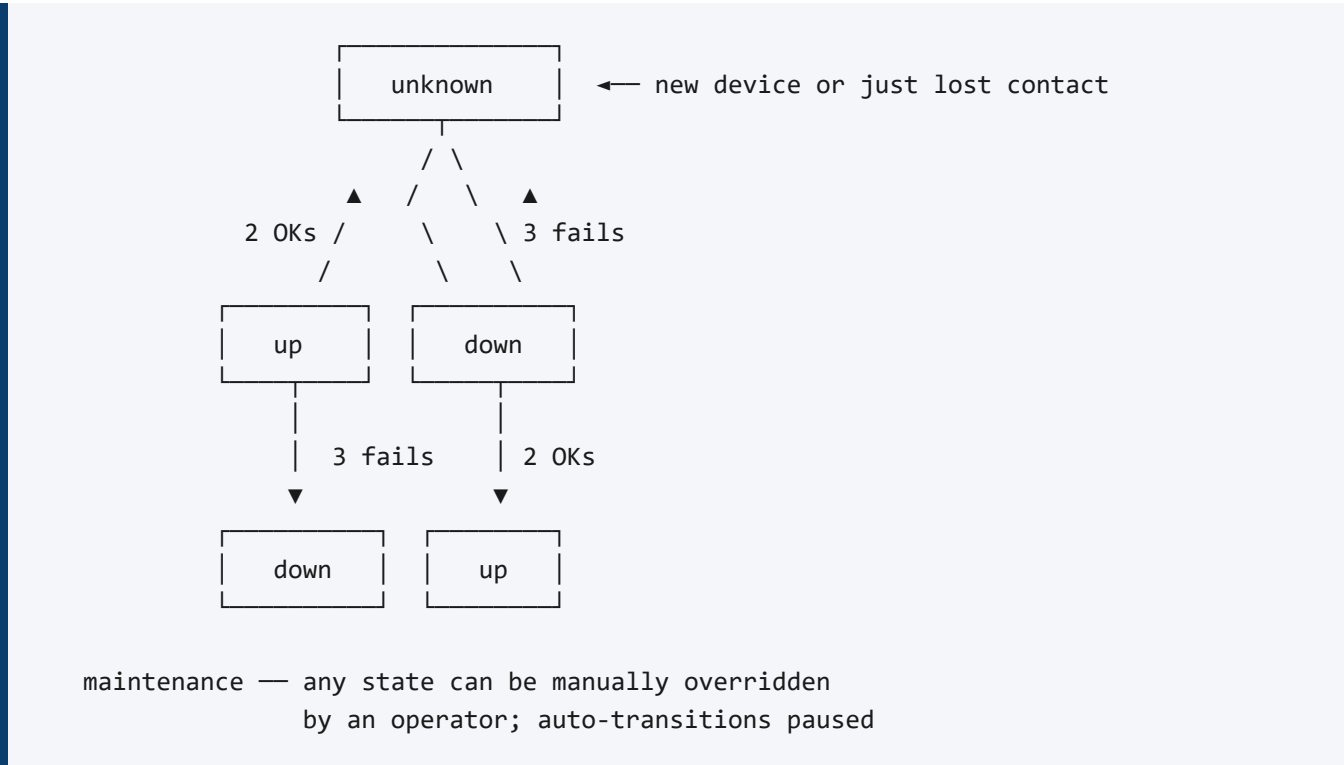
The cadence table (memorise this)

Job	Default cadence	Why
Liveness sweep (ICMP+TCP-ping)	Every 5 minutes	How fast we want to detect "device down."
SNMP interface counters poll	Every 5 minutes	For interface up/down state and bandwidth (future use).
SNMP inventory walk (LLDP, ARP, MAC, routes)	Hourly	Topology rarely changes faster than that.
Active port scan (full)	Daily, off-hours	Noisy; respect IDS and customer's working hours.
Firewall config ingest	On-change webhook + 6-hour fallback poll	Configs change rarely but importantly.
Topology snapshot	Hourly	Lines up with the inventory cadence.
Stale-job sweeper	Every 15 minutes	Mark jobs <code>timeout</code> if collector dropped mid-run.

Lifecycle sweep	Daily	Quiet, low-priority bookkeeping.
Snapshot retention (rolling-window prune)	Daily	Stops the snapshot tables from growing forever.
Audit-log archival	Weekly	Move rows older than 30 days to cold storage; keep the index.

All cadences must be per-tenant overridable — a customer with 200 sites needs longer cycles than one with 5.

The availability state machine (in detail)



Implementation pseudocode:

```
on each liveness probe result for device D:
  if probe success:
    consecutive_failures = 0
    consecutive_successes++
    if state == down and consecutive_successes >= 2:
      state = up; emit "device_recovered" event; clear active alert
  else:
    consecutive_successes = 0
    consecutive_failures++
```

```
if state == up and consecutive_failures >= 3:  
    state = down; emit "device_down" event; raise alert
```

Tunable per-tenant (some tenants want sensitive thresholds, some want quiet ones).

The alert engine (rules that matter)

Every diff or state change generates a `discovery_event` row. The alert engine's job is to decide which events become outbound webhooks and which sit silently in the feed. Default rules:

Event	Default action	Notes
<code>device_down</code> (after hysteresis)	Webhook + dashboard	The classic page.
<code>device_recovered</code>	Dashboard only	Don't wake anyone up for good news.
<code>new_device_discovered</code>	Dashboard, daily digest	Could be legit, could be a rogue. Daily summary, not real-time.
<code>device_disappeared</code> (7 days no contact)	Dashboard, weekly digest	Likely retired.
<code>device_moved_port</code> (same MAC, different switch port)	Webhook	Often interesting — laptop relocated, or worse, MAC spoofing.
<code>new_open_port_on_device</code>	Webhook for "sensitive" ports (22, 3389, 5432, 1433); dashboard otherwise	Security signal.
<code>firewall_config_changed</code>	Webhook with diff summary	Customers love this one.
<code>vpn_tunnel_dropped</code>	Webhook	Branch connectivity loss.
<code>collector_offline</code>	Webhook	If we don't hear from the collector for 5 minutes, the customer needs to know.

Two safety rails:

1. **Per-tenant rate limit.** Cap webhooks at, say, 60 per hour per tenant. Above that, batch into a digest. Prevents an outage at one customer from drowning their alerting system.

2. **Deduplication window.** Same event for same entity within 15 minutes → counter increment, not a new webhook.

Observability of our own pipeline

We expose Prometheus metrics from both the API and each collector. The minimum useful set:

Metric	Type	Why we care
<code>scan_jobs_enqueued_total{tenant, pillar}</code>	Counter	Workload visibility
<code>scan_jobs_completed_total{tenant, pillar, status}</code>	Counter	Success/failure tracking
<code>scan_job_duration_seconds{pillar}</code>	Histogram	SLO: 99% < 30 min
<code>scan_queue_depth{tenant}</code>	Gauge	Backlog detection
<code>collector_connected{tenant, collector_id}</code>	Gauge (0/1)	Collector health
<code>devices_total{tenant}</code>	Gauge	Inventory size
<code>devices_down{tenant}</code>	Gauge	Customer-visible outage size
<code>fusion_conflicts_total{rule}</code>	Counter	Quality of fact fusion
<code>discovery_events_total{kind}</code>	Counter	Tells you what your customers are seeing
<code>credential_decrypts_total{tenant}</code>	Counter	Audit anomalies

Build one Grafana dashboard that anyone in the team can open before a customer call: queue depth, error rate, collector heartbeats, top tenants by event count. It pays for itself the first time something goes wrong.

Step-by-step plan

Step 1 — Wire scheduled jobs (NestJS)

You already have `ScheduleModule.forRoot()` in `app.module.ts`. Add a class `DiscoveryScheduler` in `discovery/scheduler.service.ts` with `@Cron` decorators for each cadence above. Each handler:

1. Iterates active tenants.
2. For each, reads tenant-specific overrides.
3. Enqueues scan jobs via the dispatcher built in Chapter 0.

Step 2 — Stale-job sweeper

Every 15 min, find jobs `status=running` with `updated_at < now() - 1 hour`. Mark them `timeout`. Write an event. Don't auto-retry — the next scheduled cadence will pick up new work.

Step 3 — Availability + lifecycle workers

Two background goroutines (or NestJS providers + cron):

- **Availability worker** — runs liveness probes via collectors on the 5-minute cadence; updates state machine; emits events on transitions.
- **Lifecycle worker** — daily; promotes devices to `decommissioned` and `archived` per thresholds.

Step 4 — Alert engine

Create `discovery/alerts/` with:

- `alert-rules.entity.ts` — per-tenant configurable.
- `alert-engine.service.ts` — listens to `discovery_event` inserts (Postgres LISTEN/NOTIFY again), checks rules, applies dedup + rate limit, fires webhook.
- `webhook-dispatcher.service.ts` — uses backoff library for retry; dead-letter queue after 5 failures.

Step 5 — Metrics + structured logging

In NestJS, install `@willsoy/nestjs-prometheus` and expose `/metrics`. In the Go collector, add `github.com/prometheus/client_golang` and bind a small HTTP server on `127.0.0.1:9090` for local scraping (don't expose externally — collectors are inside customer networks). Collectors push their key metrics to the API as part of their heartbeat instead.

Migrate logs to `zap` (Go) and `nestjs-pino` (NestJS). Every log line has `tenant_id`, `collector_id`, `job_id` when available.

Step 6 — Worker pools, backoff, retries (collector)

1. Wrap each pillar (firewall ingest, SNMP walk, active scan stage) in `ants` worker pools sized per CPU and per pillar.
2. Around every external call (REST, SSH, SNMP), apply `cenkalti/backoff` with exponential delay capped at 5 minutes and $\pm 25\%$ jitter.
3. Distinguish retryable (`5xx` , network timeout) from non-retryable (`4xx` , auth failure) errors. Don't retry forever on a 401.

Step 7 — Operator dashboard

A small page in the existing Next.js app: per-tenant queue depth, last-N events, last-N scan jobs with duration, collector heartbeats. This is internal-team-only — different from the customer-facing topology.

Step 8 — Documentation, runbooks, on-call

For every alert kind, write a short *runbook*: what triggered it, where to look, how to silence it for 1 hour for maintenance windows. Three pages of runbook prevents a hundred late-night calls.

How we test Chapter 5

Test 1 — Cadence works.

1. Start the system. Wait one hour.
2. Confirm: at least one liveness sweep, one SNMP inventory, one snapshot occurred for the test tenant.
3. Each one has a `scan_job` row with `status=completed` and a non-zero duration.

Test 2 — Hysteresis prevents flapping.

1. Inject a sequence of liveness results: pass, fail, pass, fail, pass, pass.
2. State should remain `up` throughout; zero alerts emitted.

Test 3 — Real downstream alert.

1. Power off a lab device.
2. Within 15 minutes (5 min cadence + 3-strike hysteresis), webhook fires to a test endpoint.

3. Power back on; recovery event lands in dashboard but no webhook fires.

Test 4 — Stale job timeout.

1. Start a scan; kill the collector mid-run.
2. After ~15 min the sweeper marks the job `timeout`.
3. Next scheduled cadence enqueues fresh work; old job not retried.

Test 5 — Rate-limit honoured under flood.

1. Inject 1,000 `device_down` events for the same tenant in 1 minute.
2. Confirm webhook count for that minute is $\leq$ rate limit; remaining events become a digest.

Test 6 — Lifecycle automation.

1. Set a device's `last_seen_at` 8 days ago. Run lifecycle.
2. Device flips to `decommissioned`.
3. Set 95 days ago. Run again. Device flips to `archived`.
4. Plug the device in; it reappears with state `active` + `up`; an "unexpected reappearance" event is logged for security review.

Test 7 — Soak test (the real one).

1. Run the system end-to-end with a 100-device lab for 7 days.
2. Audit on day 7: zero un-handled goroutine panics, snapshot table within retention bounds, alert volume sane, collector reconnects survived (kill it briefly each day), CPU/memory of API and collector flat over the week.

Things that can go wrong (and what to watch for)

Cron-storm. All your cron handlers fire at `:00:00` and overload the system once an hour. Always add per-handler jitter (0–60 s) to spread the load.

Hysteresis too aggressive. Setting "5 misses to flip down" sounds safer but means you detect outages 25 minutes late. 3 misses with 5-minute cadence = 15 min worst case, which is a sensible default.

Webhook timeouts blocking the alert engine. Use a separate retry queue and never block the event loop on a webhook call. Backoff queue must be persistent (Postgres rows), not in-memory.

Auto-retrying on 4xx. A 401 is not a transient — your token is wrong. Retrying it 10 times in 30 minutes spams the customer's auth logs and the customer's auth team. Distinguish error classes; surface `auth_failure` as an actionable alert, not a retry storm.

Zombie observations. If a session crashes after writing observations but before fusion runs, those observations may be partial. Always fuse on session-complete only, not session-in-progress.

Snapshot table growing forever. Easy to forget the retention prune. Build it in week 1; verify table size bounded over a soak test.

Logs without tenant_id. Two months in you'll be triaging a customer issue and discover the logs don't say which customer. Add `tenant_id` to every log line as a structured field — non-negotiable.

Alerts as a substitute for visibility. If your only signal is "the alert hasn't fired," you don't know whether the system is healthy or just silently broken. Ship an "if no scans ran for tenant X in 24 h, page someone" meta-alert.

Maintenance windows ignored. Customer says "we're upgrading our firewall on Saturday 02:00–04:00." You scan during that window, the firewall reboots, you alert "device_down" 30 times. Build a maintenance-window concept in the UI; suppress availability alerts for marked devices in the window.

One-tenant overload becoming a noisy neighbour. A tenant with 5,000 devices runs full active scan and starves smaller tenants on the same collector pool. Add per-tenant queue weights and concurrency caps in the dispatcher.

Exit checklist (you can call the system production-ready when ALL are true)

- ☐ All cron cadences run for 7 days unattended without growing memory, queue, or table sizes beyond expected bounds.
- ☐ Hysteresis verified end-to-end with a real device power-off / power-on test.
- ☐ Stale-job sweeper marks `timeout` within ≤ 15 min after a collector kill.
- ☐ Webhook delivery has retry + backoff + dead-letter and verified under flood.
- ☐ Per-tenant rate limit honoured even under 1,000-event/min injection.
- ☐ Lifecycle automation moves a synthetic device through active $\rightarrow$ decommissioned $\rightarrow$ archived.
- ☐ Maintenance window suppresses alerts for marked devices.
- ☐ Operator dashboard shows queue depth, success rate, collector heartbeats, top tenants — at a glance.
- ☐ Every log line carries `tenant_id` ; metrics are scrapeable; runbooks exist for every alert kind.
- ☐ A "no-scans-in-24-h" meta-alert exists per tenant, so silence is not assumed to be health.

Recap card — Chapter 5 in 5 sentences

1. Each pillar has its own cadence: liveness 5 min, SNMP inventory hourly, active scan daily, firewall on-change + 6 h fallback, snapshots hourly.
2. Two-axis state (lifecycle $\times$ availability) with hysteresis (3 misses to flip down, 2 to flip up) prevents alert flapping.
3. The alert engine emits webhooks for events that matter, with per-tenant rate limit, dedup window, and persistent retry backoff — never block the event loop on a webhook call.
4. Prometheus metrics + Zap/Pino structured logs (always tagged with `tenant_id`) + a Grafana dashboard make silent failures impossible to hide.
5. A 7-day soak test on a real lab, plus runbooks for every alert kind and a "no-scans-in-24-h" meta-alert per tenant, is the real production-ready bar.